



Guía n.º 1.4.4

Actividad: Ejercicios de colecciones y manejo de excepciones en Java

Aplicación práctica de organización de datos, validación de operaciones y control de errores en programación orientada a objetos

Guía n.º	1.4.4
Nombre de la guía	Actividad de ejercicios sobre colecciones y manejo de excepciones en Java
Sigla	DSY1102
Asignatura	Desarrollo Orientado a Objetos
Experiencia de Aprendizaje	Analizar situaciones de almacenamiento de datos y control de errores para proponer soluciones en Java utilizando colecciones y excepciones.
Tiempo	90 minutos
Modalidad de Trabajo	Individual o en parejas
Indicadores de Logro	Selecciona colecciones adecuadas según el problema, identifica excepciones posibles y propone soluciones con validaciones y manejo de errores de forma pertinente.

Propósito de la guía

Desarrollar la capacidad de reconocer y aplicar colecciones y manejo de excepciones en Java mediante ejercicios de análisis y modelado. En esta guía no se entregan soluciones resueltas; el estudiante deberá construir sus propias propuestas a partir de un caso base y de situaciones nuevas donde sea necesario almacenar múltiples datos, evitar duplicados, asociar claves con valores y controlar errores frecuentes.

La guía toma como referencia un sistema de registro de estudiantes en un curso. A partir de ese contexto se proponen actividades para seleccionar estructuras como ArrayList, HashSet y HashMap, además de identificar, prevenir y controlar errores mediante excepciones.

Instrucciones generales

- Lee con atención el caso base antes de responder cada ejercicio.
- Distingue qué tipo de colección conviene usar según el problema planteado.
- Identifica qué operaciones pueden generar errores y qué excepción podría aparecer.
- Cuando se te pida una propuesta en Java, usa nombres coherentes con el contexto.
- Si propones clases nuevas, incluye atributos, métodos y constructor relacionados con la responsabilidad principal del objeto.
- Justifica brevemente tus decisiones de modelado y validación en cada actividad.

Formato sugerido de respuesta para cada ejercicio

1. Nombre de la clase o respuesta principal
2. Descripción breve de la responsabilidad de la clase o solución
3. Lista de atributos o elementos identificados
4. Lista de métodos o acciones relevantes
5. Constructor propuesto si corresponde



6. Justificación breve: ¿qué datos, colecciones o validaciones fueron seleccionados y por qué?

Caso base: Registro de estudiantes en un curso

Situación: Un sistema académico necesita registrar estudiantes inscritos en un curso. Se requiere almacenar nombres en orden de ingreso, evitar duplicados en ciertos listados, asociar cada estudiante con su correo institucional y controlar errores cuando se intenta acceder a un dato inexistente o registrar información no válida.

Consigna general: Observa el siguiente esquema de uso de colecciones y manejo de excepciones y utilízalo como base para desarrollar los ejercicios de análisis y aplicación en Java.

Estructura base de la clase	<pre>import java.util.ArrayList; import java.util.HashMap; import java.util.HashSet; public class Curso { private String nombre; private ArrayList<String> estudiantes; private HashSet<String> correosRegistrados; private HashMap<String, String> correoPorEstudiante; public Curso(String nombre){ this.nombre = nombre; this.estudiantes = new ArrayList<>(); this.correosRegistrados = new HashSet<>(); this.correoPorEstudiante = new HashMap<>(); } }</pre>
Ejemplo de control de error	<pre>public String obtenerEstudiante(int indice) { try { return estudiantes.get(indice); } catch (IndexOutOfBoundsException e) { return "Índice fuera de rango"; } }</pre>

Ejercicio 1: Selección de colección adecuada

Situación: Un sistema debe guardar nombres de libros en orden de llegada, evitar repetir códigos de préstamo y asociar cada estudiante con su número de matrícula.

Consigna: Indica qué colección usarías en cada caso y justifica por qué ArrayList, HashSet o HashMap sería la opción más adecuada.

Respuesta	
Explicación	
Justificación	



Ejercicio 2: Identificación de usos de ArrayList

Situación: Se desea comprender cuándo conviene trabajar con una lista ordenada de datos.

Consigna: Explica qué características de ArrayList lo vuelven apropiado para almacenar estudiantes en orden de inscripción. Menciona además una limitación que podría presentar en ciertos casos.

Respuesta	
Explicación	
Justificación	

Ejercicio 3: Análisis de HashSet

Situación: Un sistema necesita asegurarse de que un dato no se registre dos veces.

Consigna: Describe una situación del caso base donde HashSet sería más útil que ArrayList. Explica qué problema resuelve y qué ventaja aporta al diseño.

Respuesta	
Explicación	
Justificación	

Ejercicio 4: Uso de HashMap

Situación: Una aplicación académica necesita asociar el nombre de cada asignatura con su promedio final.

Consigna: Diseña una propuesta de clase que utilice un HashMap para almacenar esa relación. Explica qué claves y valores manejarías y qué métodos serían necesarios.

Clase propuesta	
------------------------	--



Atributos	
Métodos	
Constructor	
Justificación	

Ejercicio 5: Propuesta de clase con colección

Situación: Una biblioteca desea registrar varios títulos dentro de una categoría específica.

Consigna: Propón una clase `CategoriaLibros` o un nombre equivalente. Define al menos 4 atributos, 3 métodos y un constructor, incorporando una colección apropiada para guardar títulos.

Clase propuesta	
Atributos	
Métodos	
Constructor	
Justificación	

Ejercicio 6: Identificación de excepciones posibles

Situación: Un método intenta dividir dos números ingresados por el usuario, convertir texto a número y acceder a un elemento de una lista.

Consigna: Identifica qué excepciones podrían producirse en ese escenario y explica en qué momento aparecería cada una.

Respuesta	
------------------	--



Explicación	
Justificación	

Ejercicio 7: Aplicación de try-catch

Situación: Se necesita controlar errores cuando un usuario solicita un estudiante por posición dentro de una lista.

Consigna: Explica cómo usarías try-catch para evitar que el programa falle si el índice solicitado no existe. Describe además qué mensaje o acción alternativa mostrarías al usuario.

Respuesta	
Explicación	
Justificación	

Ejercicio 8: Validación con throw

Situación: Un sistema de ventas no debe permitir registrar cantidades negativas ni precios menores o iguales a cero.

Consigna: Propón un método en Java que valide esos datos y lance una excepción cuando no cumplan las reglas del problema. Explica por qué esa validación es importante.

Clase propuesta	
Atributos	
Métodos	
Constructor	
Justificación	



Ejercicio 9: Adaptación a otro contexto

Situación: Un gimnasio necesita registrar socios, evitar repetir correos y asociar cada plan con su precio mensual.

Consigna: Crea una propuesta de diseño usando al menos dos colecciones distintas. Explica qué datos almacenaría cada una y por qué fueron elegidas.

Clase propuesta	
Atributos	
Métodos	
Constructor	
Justificación	

Ejercicio 10: Mejora del diseño orientado a objetos

Situación: Se quiere fortalecer el sistema del curso para que sea más claro, seguro y reutilizable.

Consigna: Propón dos mejoras al modelo presentado. Pueden ser nuevas validaciones, más métodos, una mejor separación de responsabilidades o un uso más apropiado de colecciones y excepciones. Justifica por qué tus cambios aportarían al diseño.

Respuesta	
Explicación	
Justificación	

Criterios de evaluación sugeridos

Categoría	% logro	Descripción de niveles de logro
-----------	---------	---------------------------------

Comprensión de colecciones	25%	Reconoce diferencias entre ArrayList, HashSet y HashMap y las aplica según la necesidad del problema.
Identificación y control de excepciones	25%	Distingue errores posibles y propone mecanismos adecuados de control mediante excepciones.
Aplicación en clases y métodos	25%	Integra colecciones y validaciones dentro de propuestas coherentes de clases en Java.
Claridad y justificación	25%	Presenta respuestas ordenadas, comprensibles y bien fundamentadas según el contexto planteado.

Resumen de avance de la guía

Bloque	Cantidad de ejercicios	Producto esperado
Análisis de colecciones	5	Selección y justificación de estructuras de almacenamiento según el problema
Manejo de excepciones	3	Identificación de errores y propuestas de control mediante validaciones y try-catch
Aplicación a nuevos contextos	2	Diseño de soluciones propias usando colecciones y excepciones en Java
Total	10	10 actividades de análisis y modelado en Java

Cierre

Al finalizar esta guía, el estudiante debería ser capaz de analizar problemas donde se requiera almacenar múltiples datos y controlar errores frecuentes, seleccionando colecciones adecuadas y aplicando manejo de excepciones para proponer soluciones en Java más seguras, claras y mantenibles.